RUPEELENS

Financial Visibility & Intelligence Dashboard

Hackathon Project Report

Team name: DREAM STACK

Team members: [Shubha G S, Sinchana, Sinchana G R, Srushti Ramesh Somannavar]

Institution: BMS College of Engineering.

Submission date: 26 October 2026

Live demo: https://willowy-tiramisu-090a1a.netlify.app

Project Overview

Project title: RupeeLens — Financial Visibility & Intelligence Dashboard.

Theme: Personal finance awareness, expense visibility, and spending-pattern discovery.

Executive summary: RupeeLens is a browser-based financial dashboard designed to make personal spending easier to understand. It brings transaction entry, income-versus-expense visibility, potential expense-leak concepts, subscription tracking, a monthly forecast, and a what-if savings simulator into one interface. The goal is to help people with limited financial knowledge review their money habits and consider possible savings opportunities.

Official problem statement: Develop a solution that transforms fragmented financial activity into a clear, useful picture of an individual's spending behaviour, recurring commitments, and overall financial position. The solution should help users categorise transactions, identify recurring payments, distinguish essential commitments from discretionary spending, and recognise meaningful changes in their financial behaviour. It should account for transfers between a user's own accounts, refunds, 6irregular income, shared expenses, and other transactions that may distort a simple spending summary. Users should be able to investigate unexpected spending, estimate the future impact of recurring commitments, compare actual expenditure against a budget, and explore practical adjustments to improve their financial position. The solution should make its conclusions understandable and allow users to correct inaccurate classifications or assumptions.

1. Problem Understanding

People may lose visibility into their finances when purchases are frequent, small, spread across categories, or recurring. Reviewing transactions manually can make it difficult to notice patterns, compare planned spending with actual spending, or estimate the effect of reducing flexible expenses.

Target users

Potential users include students, young adults, and other people who want a simple overview of spending without needing advanced financial knowledge.

Needs addressed

The interface aims to bring key money indicators together, support manual transaction recording, highlight patterns worth investigating, make recurring subscriptions visible, and let users explore hypothetical spending reductions.

2. Proposed Solution and User Journey

The published interface presents an overview with total income, total expenses, remaining balance, and a potential-expense-leak count. Users can record a transaction with a description, amount, date, transaction type, category, and spending type. Additional sections provide transaction comparisons, an expense-leak radar, subscription tracking, a 12-month forecast, a savings simulator, anomaly-related insights, a savings challenge, and a Money AI chat interface.

Typical user journey: open the website → add or review transaction data → inspect the financial overview and charts → review potential expense patterns and subscriptions → compare planned and actual expenses → try a hypothetical savings adjustment.

3. Innovation and Differentiation

RupeeLens brings several financial-awareness ideas together in one interface rather than focusing only on recording expenses. The potential-expense-leak concept encourages users to investigate patterns, while the what-if simulator presents possible savings as estimates. The design goal is to make financial visibility approachable for people who may find traditional financial tools complex.

This is a prototype-level differentiation claim, not a claim that the features are unique in the market. A stronger comparison would require testing against existing tools and feedback from target users.

4. System Architecture

The current high-level architecture, based on the visible project and stated technology stack, is a client-side web application.

User
Opens the published Netlify website in a browser
HTML page structure | **CSS** styling | **JavaScript** interactions and calculations
Browser-side data handling / local storage, if confirmed in source code
Dashboard views: overview, transactions, leak radar, subscriptions, forecast, simulator, anomaly radar, savings challenge, Money AI interface

The report does not assume a backend, database, bank integration, or external API. Add those components only if verified in the submitted code. If Money AI calls an external AI service, document that dependency accurately.

5. Technology Stack

- HTML — page structure.
- CSS — layout, colours, typography, and visual design.
- JavaScript — user interactions and financial calculations.
- Netlify — hosting for the public demo.
- Browser storage — include only if verified in the implementation.

Live demo: https://willowy-tiramisu-090a1a.netlify.app

Public source repository: https://github.com/Srurv/RupeeLens.git

6. Implementation and Features

Visible section / feature	Purpose	Status to verify
Financial Overview	Income, expenses, remaining balance, potential leak count	Test totals update correctly
Record a Transaction	Enter income or expense details	Test valid and invalid inputs
Income vs Expenses	Compare finances by month/year	Test chart and filter behaviour
Where Your Money Goes	Explore spending categories	Confirm chart renders and updates
Expense Leak Radar	Show recurring costs / unusual patterns	Test detection logic with sample data

Subscription Manager	Track amount, billing frequency, next date, annual cost	Test add, delete, totals and date handling
12-Month Money Forecast	Compare planned and actual expenses	Test monthly plan saving and estimates
What-If Savings Simulator	Estimate savings after flexible-spend reduction	Check formula and range limits
Financial Anomaly Radar	Intended to flag deviations, duplicate charges, merchant spikes	Do not claim reliable detection until tested
Money Leak Detective	Shows a leak score and potential leak messages	Test with known sample transactions
Smart Savings Challenge	Displays challenge progress and badges	Confirm progress changes from actions
Rupees Lens Money AI	Chat-style financial question interface	Verify whether responses are real, rule-based, or a placeholder

7. Testing and Validation

Record the observed result for each test on the final version. Do not mark a test as passed unless it was performed.

Test	How to test	Actual result
Website access	Open the live Netlify URL in a browser	[Record]
Transaction entry	Add a valid income and expense	[Record]
Input validation	Try blank description, invalid amount, and missing date where applicable	[Record]
Test	How to test	Actual result
Dashboard totals	Compare displayed totals with manually calculated sample data	[Record]
Charts	Add transactions in different categories/months and inspect charts	[Record]
Persistence	Refresh the page and see if transactions remain	[Record]
Subscription annual cost	Add a subscription with known frequency and verify annual total	[Record]
Forecast	Add planned expenses and actual expenses for a month	[Record]
Savings simulator	Change reduction percentage and check estimated savings	[Record]
Leak/anomaly detection	Use a known sample set including repeated or unusual expenses	[Record]
Mobile layout	Open on a phone-sized screen and check readability/ controls	[Record]

8. AI, Data, and External Dependencies

List the AI tools used during ideation, coding, debugging, and report preparation. Identify external libraries, charting tools, CDNs, APIs, and hosting services by name and purpose after reviewing the HTML and JavaScript. If the Money AI interface is only a front-end mock-up or does not return live AI responses, disclose that clearly.

AI Tools:

• ChatGPT – Used for idea generation, code debugging, and report preparation.

Libraries:

• Chart.js – Used to display income, expense, and category charts.

Technologies:

• HTML5 – Structure

• CSS3 – Styling

• JavaScript – Logic and calculations

Hosting:

• Netlify – Website deployment

Version Control:

• GitHub – Source code management

APIs:

• No external APIs were used.

Datasets:

• User-entered transaction data (no external dataset).

9. User Interface and Demonstration

Insert screenshots from the actual deployed site. Recommended screenshots: (1) top overview and transaction form; (2) income-versus-expense and category charts; (3) Expense Leak Radar and Money Leak Detective; (4) Subscription Manager and forecast; (5) What-If Savings Simulator; (6) Financial Anomaly Radar, Savings Challenge, and Money AI interface.

Dashboard Overview

Transaction Entry

Income vs Expense Analysis

Hidden Expense Detection & Savings Simulator

Money AI Assistant

Live Demo:

https://jwillowy-tiramisu-

Insert 090a1a.netlify.app

Source Code:

https://github.com/yourusername/RupeeLens

10. Limitations and Incomplete Work

This is a prototype and should not be described as a bank-connected service unless a secure integration is implemented and tested. Expense-leak and anomaly indicators may be heuristic and should be presented as prompts to investigate, not proof of fraud or guaranteed savings. The Money AI feature must be described according to its actual implementation. Browser-side storage, if used, may be tied to the browser/device and is not a substitute for secure multi-user storage.

Known Limitations and Incomplete Features

- RupeeLens is currently a prototype developed for demonstration purposes.

- User data is stored locally in the browser and is not synchronized across devices.

- AI recommendations are based on simple rules and may not always provide accurate financial advice.

- Hidden expense detection and monthly forecasts are estimates based on user-entered data.

- Bank account integration, user authentication, and cloud database support are not yet implemented.

- Future updates will focus on improving prediction accuracy, security, and additional financial features.

11. Real-World Viability and Impact

A simple expense-visibility tool could help users notice spending patterns and reflect on possible savings. Before real-world deployment, it would need user testing, robust calculation validation, privacy and security review, accessible mobile design, clear explanations for insights, and safeguards against presenting uncertain anomaly flags as confirmed fraud.

12. Future Scope

Potential future improvements include more robust category rules, recurring-expense detection, editable budgets, reminders, exportable reports, clearer explanations for leak scores, accessibility improvements, secure account-based storage, and optional financial-data integrations. AI-based features should provide transparent reasoning and be evaluated for reliability.

13. Setup and Reproducibility

1. Open the public GitHub repository.

2. Download the project files or clone the repository.

3. Keep index.html, style.css, script.js, and all required assets in their expected relative locations.

4. Open index.html in a modern browser, unless the source requires a local server.

5. Use sample/fictitious financial data for demonstration.

6. Follow any additional dependency or setup instructions confirmed from the actual code.

The repository should include all relevant assets and a README with setup instructions. No API credentials or private information should be committed.

14. Team Contributions, References, and Acknowledgements

Team Name: RupeeLens

Team Members and Contributions:

Sinchana: Developed the frontend using HTML, CSS, and JavaScript, and integrated the dashboard features.
Sinchana G R: Implemented expense tracking, charts, and hidden expense detection.
Srushti Ramesh Somannavar : Worked on testing, debugging, Netlify deployment, and GitHub repository management.
Shubha G S: Prepared the project report, documentation, presentation, testing, and demonstrated the project during the hackathon.
References: List actual documentation, libraries, APIs, and learning resources used. Include the names and URLs of resources after verifying them.